



Enterprise Ontology-Driven Development

Jiri Matula^(✉) and Frantisek Hunka

University of Ostrava, Ostrava, Czechia

{jiri.matula, frantisek.hunka}@osu.cz

Abstract. Most of the current techniques and approaches for user requirements specification have problems with capturing the appropriate context for development of enterprise information systems. Primarily, they are designed to capture the functional aspects of software rather than its relevancy to an enterprise. Transactions defined in the DEMO (Design & Engineering Methodology for Organizations) represent business activities in their existential essence without implementation details. Therefore, they are great candidates to be utilized for the initial development phase of enterprise information systems. The paper exemplifies how to specify software specification the using the DEMO transaction pattern and BDD (Behaviour-Driven Development) technique. This proposal resulted from a significant lack of direct utilization of ontology for enterprise information systems development. The major part of the paper gives a step-by-step explanation of how to integrate DEMO transaction patterns into initial BDD scenarios for the development of enterprise information systems. Such created scenarios provide a perfect guideline in the initial phase of information system development for enterprises. The created scenarios were verified using the domain specific language Gherkin and BDD framework Behat.

Keywords: DEMO methodology · Behaviour-Driven development
User requirements specification · Enterprise information system

1 Introduction

Information systems (IS) provide information for agenda fulfilment of many actors working in an enterprise. The goal of an information system, in general, is to support the business processes of an organization or even to automate them completely. This fact leads to investments into implementation of an information system with expectations of competitive advantages and saving the resources.

Nevertheless, the development of information systems is a continuous process and very prone to errors and challenges. Thus, it requires very careful decisions to which functionalities are worth developing or not. Understanding needs of businesses is the foundation stone of any successful enterprise system and almost every initial phase of development is accompanied by significant issues in terms of negotiating user requirements. Moreover, user requirements often tend to be uncertain, ambiguous and usually rely on one-way confirmation.

The agile community recommends several different techniques (e.g. Use Case, User story, BDD, etc.) for the definition of user requirements and stabilizing the

development process of software [1, 2, 12]. On the one hand, these techniques work well with the general user requirements. On the other hand, they have problems with linking the context of user requirements to business activities.

The transaction pattern defined in the DEMO methodology is a general pattern which captures the essential descriptions of every business process. Essentially, this pattern is potentially the missing describing part necessary for identification information system features which are relevant or not to business processes. Despite this fact, approaches which utilize ontology for software development exist [4, 8, 17], not many of them directly control the development process which gives an opportunity to utilize ontological enterprise descriptions and combine them with BDD technique into one coherent whole. Such a unique combination of DEMO methodology and BDD technique allows the following of the latest trends of project management in software development and directly utilize enterprise ontology within development process at the same time. BDD methodology works well with a declarative approach. Therefore, this paper tries to deal with user requirements using declarative semantic. Using a declarative approach to describe an enterprise can be also found in [5–7, 9]. Also, other research papers [10, 11, 13, 14] deal with the topic by formalization of techniques based on the best practices. Nevertheless, none of them combines ontological and best practice approaches for the definition of user requirements as it is presented in this paper.

The structure of the paper is as follows. Section 2 briefly describes the embedding of transactions into BDD scenarios. Section 3, the major part of the paper, shows step-by-step how to setup an initial guideline for the development of the enterprise information system. Section 4 deals with conclusion of the presented approach.

1.1 DEMO Methodology and Transaction Pattern

DEMO (Design Engineering Methodology for Organization) defines an organization as a composition of people (social individuals) that perform two kinds of acts - production and coordination acts [3]. By performing production acts, people fulfil the aims of the organization. An example of a production fact might be e.g. “checkout of order” or “customer payment”. By performing coordination acts, human beings enter into and comply with commitments which initiate and coordinate production acts. The result of successfully performing a production act is a production fact. Coordination and production acts and facts are arranged into a transaction pattern.

The transaction pattern states that there are always two roles in a transaction, initiator (customer) and executor (producer). Initiator is someone who has a request and executor is responsible for fulfilling initiator needs. More detailed explanation of transaction pattern is depicted in Fig. 1. White boxes represent coordination acts, white rounded boxes are used for coordination facts. Production act is depicted as a grey box. Grey rounded boxes stand for production fact. Lifespan of every transaction has three phases – order (proposition), execution and result phase. In the order phase, the initiator and the executor work to reach an agreement about the intended result of the transaction, i.e., the production fact that the executor is going to create as well as the intended time of creation. In the execution phase, this production fact is brought about by the executor. In the results phase, the initiator accepts or rejects result (production fact) of the transaction [3].

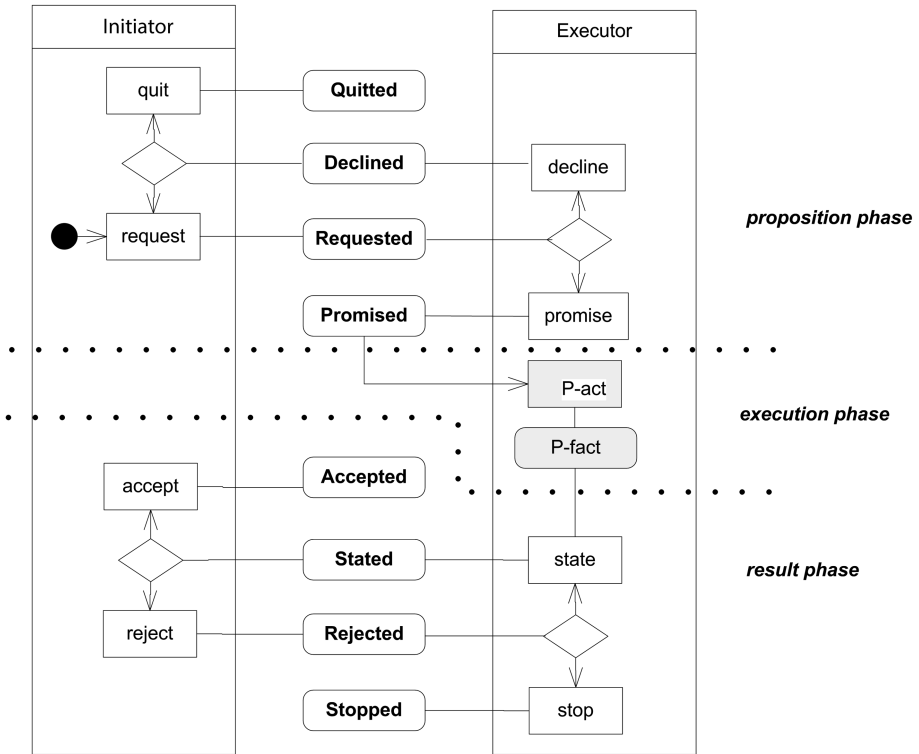


Fig. 1. Standard transaction pattern Source [3]

In simple terms, the transaction pattern provides a great opportunity of how to capture the main business processes of the enterprise in their existential essence. Usually, a business process is formed by one or more DEMO transactions arranged in a tree structure. Comparatively to other approaches which utilize ontology for business process modelling, the DEMO transaction pattern has as only one solid theoretical foundation [3].

1.2 Behaviour-Driven Development

Behaviour-Driven Development (BDD) is a set of software engineering practices designed to help teams build and deliver more valuable, higher quality software faster. It draws on Agile and lean practices including Test-Driven Development (TDD) and Domain-Driven Design (DDD). But most importantly, BDD provides a common language based on simple, structured sentences expressed in English (or in the native language of the stakeholders) that facilitate communication between project team members and business stakeholders [2]. These descriptions are called scenarios and they are being utilized for testing purposes (e.g. acceptance testing).

The BDD scenario consists of a feature title, an associated user story [1], and scenarios. Each scenario is defined by three keywords. *Given* describes context, *when* specifies actions or events and *then* states expected outcomes of the performed actions (Fig. 2).

Feature: [title]
 In order to [benefit]
 As [role]
 I want [feature]
Scenario: [title]
 Given [context]
 And [some more context]
 ...
 When [some event occurs]
 And [some other event]
 ...
 Then [outcome]
 And [some other outcome]
 ...
Scenario: [title]

Fig. 2. The BDD scenario template recommended by the agile community Source [2]

Although, BDD fits very well to modern ways of development for small and middle size information systems; its usage is rather more general than suited for the development of enterprise information system. Nevertheless, it offers an opportunity for integration of enterprise ontology (the DEMO transaction pattern), which is explained in the next chapter.

2 Embedding Transactions into BDD Scenarios

An inevitable part of the BDD scenario is the user story, typically brief description told from perspective of the person who desires the new functionality. BDD scenario typically follows a simple template in Fig. 3.

As a <type of user>, I want <some goal> so that <some reason>.

Fig. 3. The user story template recommended by the agile community Source [2]

Previously published author's papers [15, 16] have presented how to map DEMO transactions to user stories and BDD scenarios. The template in Fig. 3 was modified into the form which fits for embedding the transaction pattern within a user story. The changes to the original template are as follows:

The type of user was replaced by either initiator or executor. The part defining user intentions was exchanged by coordination or production acts which are followed by either coordination or production facts (Fig. 4).

As an <initiator/executor>, I perform a coordination/production act in <transaction> so that <coordination/production fact>.

Fig. 4. The modified user story template utilizing DEMO transaction pattern Source [15]

In other words, the ontological origin of employee activities descriptions was utilized for formalization of user stories on the theoretical foundation given in the DEMO methodology. This is the significant difference between the before mentioned techniques (Use Case, User Story, BDD, etc.) that rely only on the best practices and do not use any kind of theoretical apparatus. Such a modified user story can be consequently integrated into BDD scenario. This is depicted in Fig. 5.

Feature: [title] - [transaction ID]
In order to [production/coordination fact]
As [initiator/executor]
I want to perform [coordination/production act]
Scenario: [title]
 Given [context]
 And [some other context]
 ...
 When [some event occurs]
 And [some other event]
 ...
 Then [expected outcome or coordination/production fact]
 And [another outcome]
 ...
Scenario: [title] ...

Fig. 5. The BDD scenario template for embedding of the DEMO transaction Source [15]

3 Example Case

The description of the existing company (located in Ostrava) was chosen as the explanatory case. The second, third, and the fourth steps comes from the DEMO methodology and its general elicitation method. Understanding of the below mentioned analysis and synthesis [3] from the DEMO methodology is necessary for the clear comprehension of the described apparatus. The following chapter describes each step of the procedure.

Procedure includes the following actions:

1. Gather text description of enterprise and business activities.
2. Perform the Performa-Informa-Forma analysis.
3. Perform the Coordination-Actors-Production analysis.
4. Perform the Transaction pattern synthesis.
5. Create BDD scenarios with embedded identified transactions.
6. Generate BDD test skeletons (optional).

3.1 The Performa-Informa-Forma and Coordination-Actors-Production Analysis

In the beginning it is necessary to gather text description, respectively descriptions of employee activities in the enterprise. Text descriptions should be discussed and

confirmed by management and employees at the same time if possible. The true activities of employees might often differ from the expectations of the management board.

Gathering text description is followed by the second step – the Performa-Informa-Forma analysis. In this step, all available pieces of knowledge are divided in three sets according to the distinction axiom defined in [3]. This analysis can best be performed by colouring the appropriate parts of the descriptions: red for Performa items, green for Informa items, and blue for Forma items.

In the third step, the performa items are divided into C-acts/results, P-acts/results, and actor roles, according to the operation axiom. Actor roles are enclosed between square brackets “[” and “]”, coordination acts/results between parentheses “(” and “)”, production acts/results between the angular brackets “<” and “>”. To avoid any confusion, the enclosed pieces of text are underlined. The result of first three steps is following:

The [company] <provides> the delivery of electricity and a “smart measuring” service which enables the monitoring of electricity consumption online. In addition, metering devices have the ability to save electricity. [Customer] <has to rent> a necessary device to <start monitoring> consumption. [Customers] have provided the [information systems] (IS) which reports electricity consumption and savings for each period. Measuring devices broadcast consumption data. These records are stored for further processing to a database.

In the beginning, a [client] (calls) the company and a [salesman] (meets) a customer and <offers> him a suitable subscription plan to them. The organization structure of the salesman is arranged as multi-level organization which is supported by another [IS] which allows a salesman to manage their colleagues.

Furthermore, the [IS] provides to [salesmen] overviews about ongoing contracts and paid commissions. When a <contract is signed> by a [client], its personal details are entered to the [IS]. Consequently, the manufacturing of devices is requested. The device manufactory department has their own employees and stock of material. Upon the signing of the contract, device arrangements are complemented.

Once devices are prepared for expedition, the service department is notified about the necessity of installation contracted devices. Firstly, an installation place is examined by a [technician] who (decides) whether an installation is feasible. After that, installations of devices are (planned). Planning of device installation is a complex process which considers the availability of company cars, booking of accommodation, skills of technicians etc. Once the device is installed, a customer signs the montage sheet. When the <installation> of the device is completed, a [new client] is entered into the information system, the device is registered to the IS and contracted services are started to be <billed>. [Customers] have to <settle payments> for contracted services monthly.

Moreover, the company operates with their own [IT department] which is responsible for development of information systems and management of internal IT services. The [IT department] closely cooperates with the development department and inform about quotas for client data. Moreover, the [IT department] <manages> their own servers and [branch manager] Charles, <rents> a professional server housing and third-party services.

3.2 The Transaction Pattern Synthesis

In the fourth step, identified C-acts/facts and P-acts/facts are clustered into transactions. Essentially, each transaction must be identified by its production fact, where the transaction name and id are assigned. When transactions are stated, then the belonging coordination and production acts need to be connected to transactions. Some transactions might have missing coordination acts. This is almost always the case because descriptions are typically very incomplete. A more detailed explanation of the transaction pattern synthesis is available in [3]. The result of the synthesis is as follows.

Table 1. Identified transactions and their associated production facts

Transaction type	Associated production facts
T01 – service offer	PF01 – <i>A proper subscription plan was offered</i>
T02 – contract signing	PF02 – <i>A contract was signed</i>
T03 – appliance installation	PF03 – <i>A device was installed</i>
T04 – consumption monitoring	PF04 – <i>Monitoring of consumption started</i>
T05 – electricity delivery	PF05 – <i>Delivery of electricity started</i>
T06 – electricity subscription	PF06 – <i>A payment for the period was settled</i>
T07 – payments management	PF07 – <i>Services started to be billed upon the contract</i>
T08 – device rental	PF08 – <i>A device was rented for a period</i>
T09 – provision of IT services	PF09 – <i>Contracted services and housing were rented</i>
T10 – IT services management	PF10 – <i>IT department servers were managed</i>

Table 2. Identified transactions and their associated production acts

Transaction type	Associated production acts
T01 – service offer	PA01 – <i>Offer a proper subscription plan</i>
T02 – contract signing	PA02 – <i>Sign a contract</i>
T03 – appliance installation	PA03 – <i>Install a device</i>
T04 – consumption monitoring	PA04 – <i>Start monitoring of consumption</i>
T05 – electricity delivery	PA05 – <i>Provide delivery of electricity</i>
T06 – electricity subscription	PA06 – <i>Pay the monthly payment for electricity</i>
T07 – payments management	PA07 – <i>Initiate billing of contracted services</i>
T08 – device rental	PA08 – <i>Rent a device for consumption monitoring</i>
T09 – provision of IT services	PA09 – <i>Rent IT services and housing</i>
T10 – IT services management	PA10 – <i>Manage own servers</i>

Table 3. Identified transactions and their associated coordination facts

Transaction type	Associated coordination facts
T01 – service offer	CF01 – <i>A potential client met the salesman</i>
T02 – contract signing	–
T03 – appliance installation	CF02 – <i>A technician decided whether installation is feasible</i>

(continued)

Table 3. (continued)

Transaction type	Associated coordination facts
	CF03 – A device installation is planned
T04 – consumption monitoring	
T05 – electricity delivery	–
T06 – electricity subscription	–
T07 – payments management	–
T08 – device rental	–
T09 – provision of IT services	–
T10 – IT services management	–

Table 4. Identified transactions and their associated coordination acts

Transaction type	Associated coordination acts
T01 – service offer	CA01 – Call the company to order
T02 – contract signing	–
T03 – appliance installation	CA02 – Decide about feasibility CA03 – Plan the appliance installation
T04 – consumption monitoring	–
T05 – electricity delivery	–
T06 – electricity subscription	–
T07 – payments management	–
T08 – device rental	–
T09 – provision of IT services	–
T10 – IT services management	–

Previously, actor roles (text enclosed between square brackets in the company description) were identified in the third step. Roles were refined and generalized for modelling purposes. No person outside of a modelled domain would know who is, for example Charles, but almost everybody can understand who is branch manager in the IT department. In this way, each role is associated to a transaction which is depicted in Fig. 6. This diagram will be useful and will make the consequent preparation of BDD scenarios easier because it allows immediately finding out which actor role belongs to which transaction type.

3.3 Embedding Transactions into BDD Scenarios

Once coordination and production acts/facts are listed, the creation of BDD scenarios is straightforward. The created scenarios follow a template mentioned in Fig. 5. The “*In order to*” part can contain only coordination or production facts mentioned in Tables 1 and 3. Identified coordination or production acts defined in Tables 2 and 4 are placed into the “*I want to*” part of BDD specification. The actor role (initiator or executor) is attached to the “*As*” part. See results in Figs. 7 and 8.

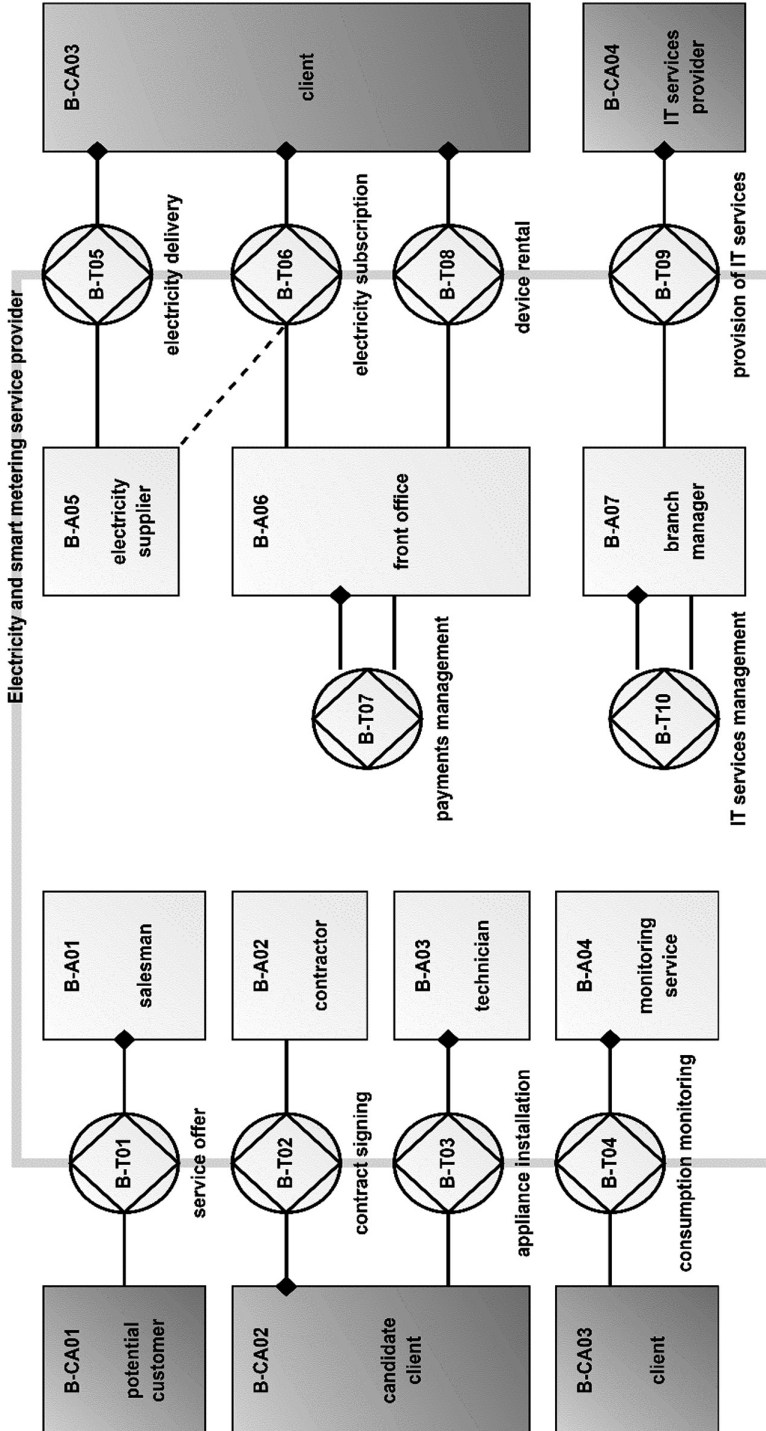


Fig. 6. The transaction diagram of the company

Feature: Service offer – T01
In order to meet with the salesman
As potential customer
I want to call the company for an order
Scenario:
Given
When
Then

Fig. 7. Embedded coordination act CA01 and coordination fact CF01 from the transaction T01

In Fig. 7 coordination act “CA01 – *call the company for an order*” was placed into “*I want to*” part of the user story. Actor role – potential customer, the initiator of transaction, was attached to the part “As”. Coordination fact “CF01 – *a potential client met the salesman*” were attached to the part “*In order to*”.

Feature: Electricity subscription – T06
In order to settle a payment for a period
As client
I want to pay month payment for electricity
Scenario:
Given
When
Then

Fig. 8. Example of embedded production act PA06 and fact PF06 from the transaction T06

In this way, all coordination and production acts and facts are embedded into BDD scenarios. Practically, the total count of created feature descriptions should be equal to the count of identified coordination and production acts. Importantly, if the certain coordination/production is completely unrelated for purposes of development then it can be omitted.

If the questionable part remains blank with scenarios within BDD specification, they might be more or less freely defined. On one hand, this is desirable because it allows the ability to describe situations for future software implementation and can capture the extra requirements of the involved stakeholders. On the other hand, they should stay coherent prior to user story/transaction given in the BDD specification. Good candidates for definitions of scenarios inside BDD feature specification are *informa* (green coloured text) and *forma* (blue coloured text) items. This is depicted in Fig. 9.

Unfortunately, the exact prescription for utilization of *forma* and *informa* items is missing so far. Utilization of knowledge retrieved from *forma* and *informa* items stays up to recommendations given in the BDD technique.

Feature: Payments management – T07
In order to start billing services upon the contract
As front office
I want to initiate billing of contracted services
Scenario:
Given device is registered in IS
When new client is entered into IS
Then contracted services are started to be billed

Fig. 9. Example of utilization of performa and forma items within BDD scenarios (Color figure online)

3.4 Generating Test Skeletons (Optional Step)

Prepared BDD scenarios are compatible with Gherkin language syntax. Gherkin is a business readable, domain specific language often used by developers who practice BDD style of development. Gherkin serves two purposes, documentation and automated tests. The crucial condition for automated generation of test files skeletons is the definition of scenario steps within a BDD feature specification. The test class skeleton for Fig. 9 was automatically generated by Behat framework. Behat is an open-source Behaviour-Driven development framework for the PHP platform. The generated output is depicted in Fig. 10.

```
class PaymentsManagementContext implements Context {

    /**
     * @Given :device is registered in IS
     */
    public function deviceIsRegisteredInIS($device)
    {
        throw new PendingException();
    }

    /**
     * @When new client is entered into IS
     */
    public function newClientIsEnteredIntoIS ()
    {
        throw new PendingException();
    }

    /**
     * @Then contracted services are started to be billed
     */
    public function contractedServicesAreStartedToBeBilled()
    {
        throw new PendingException();
    }
}
```

Fig. 10. Generated test skeleton for the BDD scenario in Fig. 9

In this manner, it is possible to utilize ontological description directly for the definition of testing steps (in form of classes in the object-oriented paradigm) and thereby connect production codebase with the ontology of enterprise.

4 Conclusions

The main intention of the paper was to present the step-by-step process on how to integrate ontological enterprise description directly into a development process for small and middle size enterprise information systems. The proposed technique is derived upon the theoretical basis of DEMO methodology and follows one of the latest trends in the software development – BDD practices. Such a combination is different from other similar approaches e.g. Use Case, User Story, BPMN and others. These techniques usually rely only on the one-way confirmation of involved stakeholders.

The presented solution provides a solid guideline, emerging from existent business processes for the definition of user requirements in the initial phase of enterprise information system development. It enables simple observation of which business activities are involved in the information system. Another important benefit is a minimalization of potential between business needs and implemented information system in its beginning, facilitated by linkage of software specifications and ontological descriptions of enterprise.

The described procedure consists of several steps. At first, it is necessary to gather text descriptions of a modelled enterprise. Then identification follows of actors, coordination/production act and facts which is known as Coordination-Actors-Production analysis. This analysis is followed by the transaction pattern synthesis. After synthesis, it is possible to set an initial guideline for development in the form of BDD scenarios which has embedded identified transactions, respectively their coordination/production acts and facts. The whole procedure was demonstrated step-by-step in the existing company in Ostrava together with reference to book Enterprise Ontology, written by Dietz [3]. Usability of created scenarios for BDD testing was verified using Behat BDD framework, whereas test files skeletons for consequent implementation was possible to generate.

To summarize, the method offers a possibility of how to directly utilize ontological descriptions of business processes during the initial phase of enterprise information system development. Despite promising qualities, it would be appropriate to verify the method once again with an adequate case study, to eventually discover so far unknown further benefits and weaknesses. Also, another step would be a proper formalization of inform and form items utilization for BDD scenarios. These mentioned facts will be an objective of further research.

Acknowledgements. The paper was supported by the grant provided by Ministry of Education, Youth and Sports Czech Republic, reference no. SGS05/PRF/2018 and the grant provided by city of Ostrava.

References

1. Cohn, M.: User Stories Applied: for Agile Software Development, pp. 31–41. Addison-Wesley, Boston (2004)
2. Smart, J.F.: BDD in Action: Behavior-Driven Development for the Whole Software Lifecycle, pp. 3–32. Manning Publications Company, New York (2014)
3. Dietz, J.L.: Enterprise Ontology: Theory and Methodology. Springer, New York (2006). <https://doi.org/10.1007/3-540-33149-2>. pp. 16–160
4. Van Kervel, S.J.H.: Ontology driven enterprise information systems engineering. Doctoral dissertation, SIKS Dissertation series nr. 2012-50, Delft University of Technology (2012)
5. Pesic, M., Van der Aalst, W.M.: A declarative approach for flexible business processes management. In: Eder, J., Dustdar, S. (eds.) BPM 2006. LNCS, vol. 4103, pp. 169–180. Springer, Heidelberg (2006). https://doi.org/10.1007/11837862_18
6. Grosz, B.N., Labrou, Y., Chan, H.Y.: A declarative approach to business rules in contracts: courteous logic programs in XML. In: Proceedings of the 1st ACM Conference on Electronic Commerce, pp. 68–77 (1999)
7. de Leoni, M., Maggi, F.M., van der Aalst, W.M.: An alignment-based framework to check the conformance of declarative process models and to pre-process event-log data. *Inf. Syst.* **47**, 258–277 (2015)
8. Skjæveland, M.G., Giese, M., Hovland, D., Lian, E.H., Waaler, A.: Engineering ontology-based access to real-world data sources. *Web Semant.: Sci. Serv. Agents World Wide Web* **33**, 112–140 (2015)
9. Gigante, G., Gargiulo, F., Ficco, M.: A semantic driven approach for requirements verification. In: Camacho, D., Braubach, L., Venticinque, S., Badica, C. (eds.) Intelligent Distributed Computing VIII. SCI, vol. 570, pp. 427–436. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-10422-5_44
10. Wautelet, Y., Heng, S., Hintea, D., Kolp, M., Poelmans, S.: Bridging user story sets with the use case model. In: Link, S., Trujillo, J. (eds.) ER 2016. LNCS, vol. 9975, pp. 127–138. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-47717-6_11
11. Simões, D., Antunes, P., Cranefield, J.: Enriching knowledge in business process modelling: a storytelling approach. In: Razmerita, L., Phillips-Wren, G., Jain, L. (eds.) Innovations in Knowledge Management, vol. 95, pp. 241–267. Springer, Heidelberg (2016). https://doi.org/10.1007/978-3-662-47827-1_10
12. Maleki, N.G., Ramsin, R.: Agile web development methodologies: a survey and evaluation. In: Lee, R. (ed.) SERA 2017. SCI, vol. 722, pp. 1–25. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-61388-8_1
13. Borgianni, Y., Cascini, G., Rotini, F.: Business process reengineering driven by customer value: a support for undertaking decisions under uncertainty conditions. *Comput. Indus.* **68**, 132–147 (2015)
14. Smit, K., Zoet, M., Berkhout, M.: Functional Requirements for Business Rules Management Systems (2017)
15. Zacek, J., Matula, J., Hunka, F.: Context definition for BDD scenarios upon DEMO methodology. In: 2nd International Conference on Theory and Practice, pp. 164–169. Sia Pacific Institute of Advanced Research, Melbourne (2016)
16. Matula J., Zacek J., Hunka F.: Relevant user stories by using DEMO analysis. In: Proceedings of the 11th Scientific Conference Internet in the Information Society, pp. 21–30. University of Dabrowa Górnicza, Cieplaka (2016)
17. Pan, J.Z., Staab, S., Almann, U., Ebert, J., Zhao, Y.: Ontology-Driven Software Development. Springer, Heidelberg (2012). <https://doi.org/10.1007/978-3-642-31226-7>